Lab 3: Academic GPA Management System

Course: Web Development

Level: 2nd Year of LMD in Computer Science

ADMIN PROFESSOR STUDENT

1 Introduction

In this lab you will build a complete **Academic GPA Management System** — a three-role web application modelling a real academic institution.

- The ADMIN sets up the entire academic structure: semesters, courses, professors, students, and enrollments.
- The PROFESSOR selects a semester and an assigned course, then enters grades for the enrolled students via an AJAX-powered interface.
- The STUDENT logs in and consults their grades and GPA history — read-only, no data entry.

The system follows a **simplified MVC** pattern. Admin operations use standard PHP POST/redirect. Only grade entry (professor) and GPA display (student) use a **REST API** with jQuery AJAX.

Responsibility	ADMIN	PROFESSOR	STUDENT
Manage semesters	✓	×	×
Manage courses (linked to semester)	✓	×	×
Manage professor accounts	✓	×	×
Manage student accounts	✓	×	×
Enroll students in semester (checkbox list)	✓	×	×
Assign professor to course + semester	✓	×	×
Select semester → course → enter grades	×	✓	×
View own grades + GPA per semester	×	×	✓
View GPA history + export CSV	×	×	✓

2 Architecture Overview

2.1 MVC Pattern

Model — PHP classes in `models/` handle all database queries using PDO prepared statements. No SQL is written outside model files.

View — PHP/HTML files in `views/` handle presentation only. They receive data via variables — no logic, no queries inside views.

Controller — PHP classes in `controllers/` handle requests, call models, and include the appropriate view.

> The Front controller — `index.php` routes every request using the `?page=` parameter.

2.2 Folder Structure

```
project/
  index.php                    <- Front controller (?page= routing)
  config.php                   <- PDO connection + requireRole() helper
  models/
    User.php                   <- User queries (all roles)
    Semester.php               <- Semester CRUD
    Course.php                 <- Course CRUD + semester link
    Enrollment.php             <- Student <-> Semester
    Assignment.php             <- Professor <-> Course <-> Semester
    Grade.php                  <- Grade upsert + retrieval
    GPA.php                    <- Weighted GPA computation
  controllers/
    AuthController.php         <- Login / logout / session guard
    AdminController.php        <- All admin actions (POST/redirect)
    ProfessorController.php    <- Grade entry page
    StudentController.php      <- Grade view + history
  views/
    login.php
    admin/
      dashboard.php            <- Summary cards
      semesters.php
      courses.php
      professors.php
      students.php
      enrollments.php          <- Checkbox list per semester
    professor/
      grades.php               <- Dropdowns + dynamic student table
    student/
      dashboard.php            <- Current semester grades
      history.php              <- All semesters + Chart.js
  api/
    grades.php                 <- Professor: GET students, POST grades
    gpa.php                    <- Student: GET grades + GPA + export
  public/
    css/style.css
    js/
      professor.js             <- AJAX grade entry
      student.js               <- AJAX GPA display
```

Listing 1: Project folder structure

2.3 Pseudo-code: Front Controller

Pseudo-code — `index.php` (front controller)

```
session_start()
page ← $_GET['page'] ?? 'login'
match page:
    'login', 'logout' → new AuthController
    starts with 'admin.' → requireRole(admin); new AdminController
    starts with 'professor.' → requireRole(professor); new ProfessorController
    starts with 'student.' → requireRole(student); new StudentController
    default → redirect to ?page=login
```

3 Database Schema

3.1 Relationships at a Glance

users (role column distinguishes admin / professor / student)
semesters $\xrightarrow{1-*}$ **courses**
users (student) $\xleftrightarrow{\text{enrollments}}$ **semesters** (many-to-many)
users (professor) $\xleftrightarrow{\text{assignments}}$ **courses** + **semesters** (many-to-many)
grades: unique on (student_id, course_id, semester_id)
gpa_records: unique on (student_id, semester_id) — recomputed on every grade save

3.2 SQL Definitions

```sql
CREATE TABLE users (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  email      VARCHAR(150) NOT NULL UNIQUE,
  password   VARCHAR(255) NOT NULL,
  role       ENUM('admin','professor','student') NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Listing 2: users

```sql
CREATE TABLE semesters (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  label         VARCHAR(20) NOT NULL,     -- e.g.  S1, S2
  academic_year VARCHAR(20) NOT NULL,     -- e.g.  2024/2025
  is_active     BOOLEAN DEFAULT FALSE,
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE courses (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  semester_id INT NOT NULL,
  name        VARCHAR(150) NOT NULL,
  credits     INT NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (semester_id) REFERENCES semesters(id)
);
```

Listing 3: semesters and courses

```sql
CREATE TABLE enrollments (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  student_id  INT NOT NULL,
  semester_id INT NOT NULL,
  enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_enroll (student_id, semester_id),
  FOREIGN KEY (student_id)  REFERENCES users(id),
  FOREIGN KEY (semester_id) REFERENCES semesters(id)
);

CREATE TABLE assignments (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  professor_id INT NOT NULL,
  course_id    INT NOT NULL,
  semester_id  INT NOT NULL,
  assigned_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_assign (professor_id, course_id, semester_id),
  FOREIGN KEY (professor_id) REFERENCES users(id),
```

```sql
  FOREIGN KEY (course_id)    REFERENCES courses(id),
  FOREIGN KEY (semester_id)  REFERENCES semesters(id)
);
```

Listing 4: enrollments and assignments

```sql
CREATE TABLE grades (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  student_id    INT NOT NULL,
  course_id     INT NOT NULL,
  semester_id   INT NOT NULL,
  professor_id  INT NOT NULL,
  grade         DECIMAL(3,1) NOT NULL,    -- 0.0 to 4.0
  entered_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uq_grade (student_id, course_id, semester_id),
  FOREIGN KEY (student_id)   REFERENCES users(id),
  FOREIGN KEY (course_id)    REFERENCES courses(id),
  FOREIGN KEY (semester_id)  REFERENCES semesters(id),
  FOREIGN KEY (professor_id) REFERENCES users(id)
);

CREATE TABLE gpa_records (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  student_id   INT NOT NULL,
  semester_id  INT NOT NULL,
  gpa          DECIMAL(4,2) NOT NULL,
  computed_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
               ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uq_gpa (student_id, semester_id),
  FOREIGN KEY (student_id)  REFERENCES users(id),
  FOREIGN KEY (semester_id) REFERENCES semesters(id)
);
```

Listing 5: grades and gpa_records

4 Step 1 — Authentication & Session Management

4.1 Overview

One login page serves all three roles. After verifying credentials, `AuthController` stores `user_id`, `role`, and `name` in `$_SESSION` and redirects to the matching dashboard.

Pseudo-code — `AuthController::login()`

```
if request is POST then
    email ← sanitize($_POST['email'])
    password ← $_POST['password']
    user ← User::findByEmail(email)
    if user exists and password_verify(password, user[password]) then
        session_start()
        $_SESSION['user_id'] ← user[id]
        $_SESSION['role'] ← user[role]
        $_SESSION['name'] ← user[name]
        $_SESSION['last_activity'] ← time()
        match user[role]:
          admin → redirect ?page=admin.dashboard
          professor → redirect ?page=professor.grades
```

```
        student → redirect ?page=student.dashboard
    else
        flash error: "Invalid email or password"
        redirect to ?page=login
    end if
else
    include views/login.php
end if
```

Pseudo-code — `requireRole($expected)` (defined in `config.php`)

```
if $_SESSION['role'] not set or time() − $_SESSION['last_activity'] > 1800 then
    session_destroy()
    redirect to ?page=login; exit
end if
if $_SESSION['role'] ≠ $expected then
    http_response_code(403)
    display "Access Denied"; exit
end if
$_SESSION['last_activity'] ← time()                    ▷ reset timeout
```

5 Step 2 — Admin: Semester & Course Management

All admin actions use standard HTML form POST and redirect back with a Bootstrap flash message. No AJAX is required in admin views.

5.1 Semester Management

Pseudo-code — `AdminController::saveSemester()`

```
requireRole(admin)
label ← sanitize($_POST['label'])
year ← sanitize($_POST['academic_year'])
id ← $_POST['id'] ?? null
if id is null then
    Semester::create(label, year)
else
    Semester::update(id, label, year)
end if
flash success; redirect to ?page=admin.semesters
```

Pseudo-code — `AdminController::toggleSemester()`

```
requireRole(admin)
id ← intval($_POST['id'])
Semester::setAllInactive()                ▷ UPDATE semesters SET is_active = 0
Semester::setActive(id)        ▷ UPDATE semesters SET is_active = 1 WHERE id = ?
flash success; redirect
```

Pseudo-code — `AdminController::deleteSemester()`

```
requireRole(admin)
id ← intval($_POST['id'])
if Course::countBySemester(id) > 0 then
    flash error: "Cannot delete: semester has linked courses"
    redirect; exit
end if
Semester::delete(id)
flash success; redirect
```

5.2 Course Management

Pseudo-code — `AdminController::saveCourse()`

```
requireRole(admin)
name ← sanitize($_POST['name'])
credits ← intval($_POST['credits'])
semId ← intval($_POST['semester_id'])
id ← $_POST['id'] ?? null
if credits ≤ 0 then
    flash error: "Credits must be a positive integer"; redirect; exit
end if
if id is null then
    Course::create(name, credits, semId)
else
    Course::update(id, name, credits, semId)
end if
flash success; redirect to ?page=admin.courses
```

Pseudo-code — `AdminController::deleteCourse()`

```
requireRole(admin)
id ← intval($_POST['id'])
if Grade::countByCourse(id) > 0 then
    flash error: "Cannot delete: grades exist for this course"
    redirect; exit
end if
Course::delete(id)
flash success; redirect
```

6 Step 3 — Admin: Professor Management & Assignment

Pseudo-code — `AdminController::saveProfessor()`

```
requireRole(admin)
name ← sanitize($_POST['name'])
email ← sanitize($_POST['email'])
pass ← $_POST['password']
id ← $_POST['id'] ?? null
if User::emailExists(email, excludeId: id) then
    flash error: "Email already in use"; redirect; exit
```

```
end if
if id is null then
    hash ← password_hash(pass, PASSWORD_BCRYPT)
    User::create(name, email, hash, role: professor)
else
    User::update(id, name, email)
    if pass not empty then
        User::updatePassword(id, password_hash(pass, PASSWORD_BCRYPT))
    end if
end if
flash success; redirect to ?page=admin.professors
```

Pseudo-code — `AdminController::saveAssignment()`

```
requireRole(admin)
profId ← intval($_POST['professor_id'])
courseId ← intval($_POST['course_id'])
semId ← intval($_POST['semester_id'])
if Assignment::courseAlreadyAssigned(courseId, semId) then
    flash error: "This course already has a professor for this semester"
    redirect; exit
end if
Assignment::create(profId, courseId, semId)
flash success; redirect
```

7 Step 4 — Admin: Student Management & Enrollment

7.1 Student Accounts

Student account creation and update follow the same pseudo-code as `saveProfessor()` above, with `role = student`. For deletion, the controller first removes grades, GPA records, and enrollments belonging to the student, then deletes the user row — always after an explicit confirmation prompt in the view.

7.2 Enrollment via Checkbox List

This is the key usability simplification of the admin interface. The admin opens a student's enrollment page and sees a list of all semesters with a checkbox next to each one (pre-checked if already enrolled). One click on **Save Enrollments** sends the full state in a single POST.

Pseudo-code — `AdminController::saveEnrollments()`

```
requireRole(admin)
studentId ← intval($_POST['student_id'])
newIds ← $_POST['semester_ids'] ?? []                    ▷ checked checkboxes
currentIds ← Enrollment::getSemesterIds(studentId)
toAdd ← newIds \ currentIds
toRemove ← currentIds \ newIds
for each semId in toAdd do
    Enrollment::create(studentId, semId)
end for
for each semId in toRemove do
    if Grade::countByStudentSemester(studentId, semId) > 0 then
```

```
        skip — add warning to flash message
    else
        Enrollment::delete(studentId, semId)
    end if
end for
flash success (with warnings if any skipped); redirect
```

8 Step 5 — Professor: Grade Entry (AJAX)

8.1 Workflow

The professor dashboard is a single page with two dropdowns and a dynamically rendered table. No page reload occurs at any step.

1. Page loads → semester dropdown populated server-side (only semesters with at least one assignment for this professor).
2. Professor selects a semester → jQuery calls `GET api/grades.php?action=courses&semester_id=X` → course dropdown updated.
3. Professor selects a course → jQuery calls `GET api/grades.php?action=students&semester_id=X&course_i` → student table rendered with one grade dropdown per row.
4. Professor fills grades, clicks **Save Grades** → jQuery calls `POST api/grades.php` → Bootstrap alert shown with count of saved grades.

8.2 API Endpoint: `api/grades.php`

Pseudo-code — GET action=students

```
requireRole(professor)
profId ← $_SESSION['user_id']
semId ← intval($_GET['semester_id'])
courseId ← intval($_GET['course_id'])
if NOT Assignment::exists(profId, courseId, semId) then
    json_encode(['error' → 'Forbidden'])
    http_response_code(403); exit
end if
students ← Enrollment::getStudentsBySemester(semId)
for each s in students do
    s[grade] ← Grade::get(s[id], courseId, semId)        ▷ null if not yet entered
end for
echo json_encode(students)
```

Pseudo-code — POST action=save

```
requireRole(professor)
profId ← $_SESSION['user_id']
semId ← intval($_POST['semester_id'])
courseId ← intval($_POST['course_id'])
entries ← $_POST['grades']                    ▷ array of {student_id, grade}
if NOT Assignment::exists(profId, courseId, semId) then
    json 403; exit
end if
saved ← 0
```

```
for each entry in entries do
    if entry[grade] NOT IN {0.0, 1.0, 2.0, 3.0, 4.0} then
        continue                                        ▷ skip invalid value silently
    end if
    Grade::upsert(entry[student_id], courseId, semId, profId, entry[grade])   ▷ INSERT ... ON DUPLICATE KEY UPDATE
    GPA::recompute(entry[student_id], semId)
    saved += 1
end for
echo json_encode(['success' → true, 'saved' → saved])
```

8.3 GPA Computation

Pseudo-code — `GPA::recompute($studentId, $semId)`

```
rows ← Grade::getAllWithCredits(studentId, semId)      ▷ JOIN grades with courses
totalPoints ← 0;    totalCredits ← 0
for each row in rows do
    totalPoints += row[grade] × row[credits]
    totalCredits += row[credits]
end for
if totalCredits > 0 then
    gpa ← totalPoints ÷ totalCredits
    GpaRecord::upsert(studentId, semId, round(gpa, 2))
end if
```

$$\text{GPA} = \frac{\sum_{i=1}^{n} (\text{grade}_i \times \text{credits}_i)}{\sum_{i=1}^{n} \text{credits}_i}$$

8.4 jQuery Client (`professor.js`)

```javascript
$(document).ready(function () {

  // Step 1: semester selected -> load courses
  $('#semesterSelect').change(function () {
    var semId = $(this).val();
    $.get('api/grades.php',
      { action: 'courses', semester_id: semId },
      function (data) {
        var opts = '<option value="">-- Select course --</option>';
        $.each(data, function (i, c) {
          opts += '<option value="' + c.id + '">'
                + c.name + '</option>';
        });
        $('#courseSelect').html(opts).prop('disabled', false);
        $('#gradeTable').hide();
      }, 'json');
  });

  // Step 2: course selected -> load students
  $('#courseSelect').change(function () {
    var semId    = $('#semesterSelect').val();
    var courseId = $(this).val();
```

```
    $.get('api/grades.php',
      { action: 'students', semester_id: semId, course_id: courseId },
      function (students) {
        var html = '<tbody>';
        $.each(students, function (i, s) {
          var gradeVal = s.grade !== null ? s.grade : '';
          html += '<tr>'
                + '<td>' + s.name + '</td>'
                + '<td>' + s.id + '</td>'
                + '<td><select name="grade_' + s.id + '" '
                +         'data-student="' + s.id + '" '
                +         'class="form-control grade-input">'
                + buildOptions(gradeVal)
                + '</select></td></tr>';
        });
        html += '</tbody>';
        $('#gradeTable').find('tbody').replaceWith(html);
        $('#gradeTable').show();
      }, 'json');
  });

  // Step 3: save grades
  $('#saveBtn').click(function () {
    var semId    = $('#semesterSelect').val();
    var courseId = $('#courseSelect').val();
    var grades   = [];
    $('.grade-input').each(function () {
      grades.push({
        student_id: $(this).data('student'),
        grade:      $(this).val()
      });
    });
    $.post('api/grades.php',
      { action: 'save', semester_id: semId,
        course_id: courseId, grades: grades },
      function (res) {
        var cls = res.success ? 'alert-success' : 'alert-danger';
        var msg = res.success
                ? res.saved + ' grade(s) saved.'
                : res.error;
        $('#feedback').html(
          '<div class="alert ' + cls + '">' + msg + '</div>');
      }, 'json');
  });

  function buildOptions(selected) {
    var opts = [['','-- Grade --'],['4.0','A'],['3.0','B'],
                ['2.0','C'],['1.0','D'],['0.0','F']];
    return opts.map(function(o) {
      return '<option value="' + o[0] + '"'
           + (o[0] == selected ? ' selected' : '') + '>'
           + o[1] + '</option>';
    }).join('');
  }

});
```

Listing 6: `professor.js` — AJAX grade entry

9 Step 6 — Student: Grades & History (AJAX)

9.1 Dashboard — Current Semester

On page load, `student.js` calls `GET api/gpa.php?action=current`. The API returns the active semester's courses with grades and the computed GPA. Courses with no grade yet show *"Pending"*.

Pseudo-code — `api/gpa.php` (action: current)

```
requireRole(student)
studentId ← $_SESSION['user_id']
semester ← Semester::getActive()
if semester is null then
    echo json_encode(['error' → 'No active semester']); exit
end if
if NOT Enrollment::exists(studentId, semester[id]) then
    echo json_encode(['error' → 'Not enrolled']); exit
end if
courses ← Course::getBySemester(semester[id])
for each course in courses do
    course[grade] ← Grade::get(studentId, course[id], semester[id])
    course[grade_points] ← (course[grade] ?? 0) × course[credits]
end for
gpa ← GpaRecord::get(studentId, semester[id])[gpa] ?? null
echo json_encode([ 'semester' → semester, 'courses' → courses, 'gpa' → gpa ])
```

9.2 GPA Color Coding

GPA Range	Interpretation	Bootstrap Alert Class
≥ 3.7	Distinction	`alert-success`
≥ 3.0	Merit	`alert-info`
≥ 2.0	Pass	`alert-warning`
< 2.0	Fail	`alert-danger`

9.3 History Page

Pseudo-code — `api/gpa.php` (action: history)

```
requireRole(student)
studentId ← $_SESSION['user_id']
semesters ← Enrollment::getSemestersByStudent(studentId)
for each sem in semesters do
    sem[courses] ← Course::getBySemester(sem[id])
    for each course in sem[courses] do
        course[grade] ← Grade::get(studentId, course[id], sem[id])
        course[grade_points] ← (course[grade] ?? 0) × course[credits]
    end for
    sem[gpa] ← GpaRecord::get(studentId, sem[id])[gpa] ?? null
end for
echo json_encode(semesters)                    ▷ used by Chart.js line chart
```

9.4 CSV Export

```php
case 'export':
  requireRole('student');
  $studentId = $_SESSION['user_id'];
  $rows = Grade::getAllWithDetailsByStudent($studentId);
  header('Content-Type: text/csv');
  header('Content-Disposition: attachment; '
       . 'filename="gpa_history.csv"');
  $out = fopen('php://output', 'w');
  fputcsv($out,
    ['Semester','Academic Year','Course','Credits',
     'Grade','Grade Points']);
  foreach ($rows as $row) {
    fputcsv($out, [
      $row['semester_label'],
      $row['academic_year'],
      $row['course_name'],
      $row['credits'],
      $row['grade'],
      $row['grade'] * $row['credits'],
    ]);
  }
  fclose($out);
  exit;
```

Listing 7: `api/gpa.php` — CSV export (action=export)

10 Step 7 — REST API Summary

Only two API files handle all AJAX communication.

File	action	Method	Description
`api/grades.php`	`courses`	GET	Professor's assigned courses for a semester
`api/grades.php`	`students`	GET	Enrolled students + existing grades for a course
`api/grades.php`	`save`	POST	Save grade batch; recompute GPA
`api/gpa.php`	`current`	GET	Active semester grades + GPA (student)
`api/gpa.php`	`history`	GET	All semesters grades + GPA (student)
`api/gpa.php`	`export`	GET	Stream CSV download of student's full history

API security rules applied to every endpoint:

1. Call `requireRole()` before any other logic.
2. Set header `Content-Type: application/json`.
3. Return HTTP `401` if session is missing, `403` if role or ownership check fails, `400` if input validation fails.
4. Never return an HTML error page from an API endpoint.

11 Step 8 — Security & Validation

1. **Prepared statements everywhere** — all model methods use PDO with `execute([...])` or `bindParam()`. No SQL string concatenation anywhere in the codebase.

2. **Output escaping** — all values rendered in views are wrapped with `htmlspecialchars($val, ENT_QUOTES, 'UTF-8')` before being echoed.
3. **Role check on every entry point** — controllers call `requireRole()` first; API endpoints call it before any other logic. This is non-negotiable and never skipped.
4. **Ownership check in APIs** — the professor API verifies the professor is assigned to the requested course before returning data or saving. The student API verifies the data belongs to the logged-in student.
5. **Protected deletes** — before any `DELETE` query, the controller checks for linked records and returns a descriptive flash error if deletion would violate referential integrity.
6. **Password security** — hashed with `PASSWORD_BCRYPT`, never logged or echoed; temporary passwords communicated out-of-band.
7. **Session timeout** — `last_activity` timestamp updated on each request; session destroyed after 1800 seconds of inactivity.

12 Step 9 — Homework

Extend the system with one or more of the following features:

- **Admin dashboard** — summary cards (total students, professors, semesters) and a Chart.js bar chart of average GPA per semester across all students.
- **Professor grade export** — allow the professor to download the grade sheet of their selected course as a CSV, using the same streaming technique shown in Section 8.4.
- **Change password page** — available to all three roles. The user enters their current password and a new one; the controller verifies the current password with `password_verify()` before updating.
- **Pagination** — page-based pagination on all admin list views using `LIMIT`/`OFFSET` in the model and a Bootstrap pagination component in the view.
- **Search & filter** — a search bar on the admin student and professor lists (filter by name or email), and a semester filter on the courses and enrollments pages.
- **GPA line chart** — on the student history page, render a Chart.js line chart of the student's GPA across enrolled semesters, using data returned by `api/gpa.php?action=history`.
- **Flash message system** — a reusable `flash($type, $msg)` helper that stores a message in `$_SESSION` and renders it as a dismissible Bootstrap alert on the next page load, then clears it.

> *Be creative and feel free to add any extra features you find interesting!*